

ENGN 1931Y: Control Systems Engineering

Milan Capoor

Fall 2026

1 Sept 10

Control system: a mechanism that alters the future state of a system

Control theory: a strategy to achieve the desired system performance

- Alternatively, the study of how to regulate and manage the behavior of dynamic system using feedback control loop

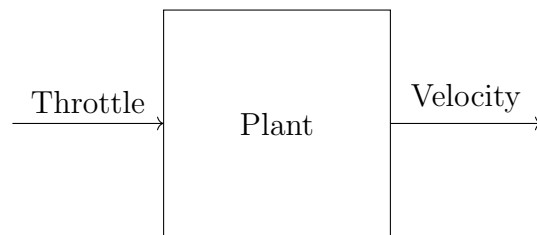
2 Feb 15

Types of Control Systems

- Logic control (e.g. programmable logic controller)
- On-off control (e.g. light switch)
- Feedback control

In this class we will focus on feedback control systems and represent almost everything with block diagrams abstracting the system into inputs and outputs.

Example: A car block diagram

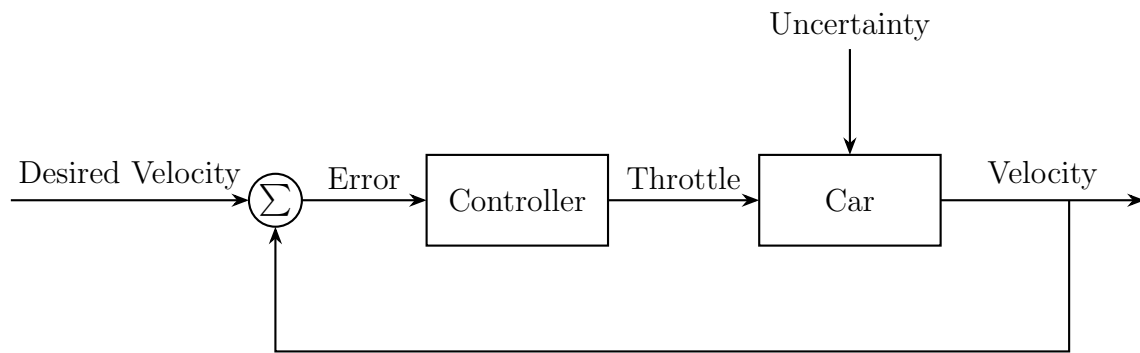


Open loop control: system without feedback.

- In the car example, we can imagine a cruise control system that sets the throttle to a fixed value. Without incorporating knowledge of dynamics or feedback, we cannot compensate for uncertain masses, varying environmental controls, or imprecise models

Closed loop control: system with feedback used to control despite our incomplete knowledge (though this trades off stability, response speed, etc.)

- In the car example, we update the throttle command based on a measurement of the current speed

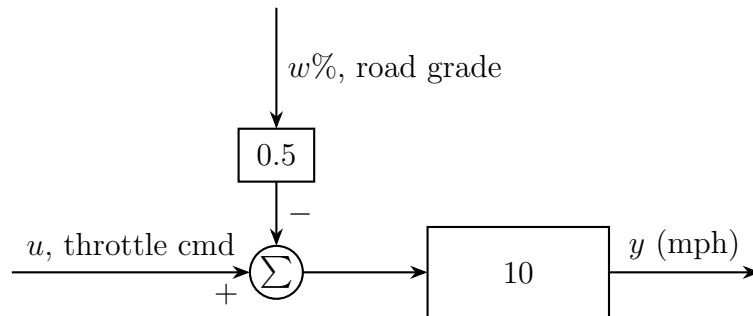


Any block diagram must involve:

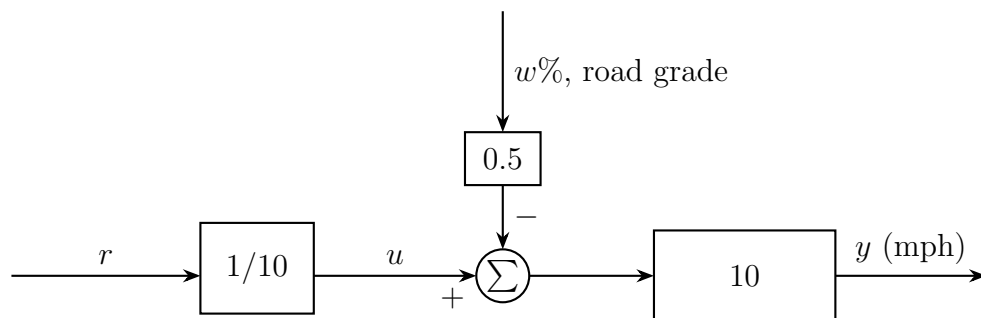
- Desired reference
- Controller
- Plant
- Sensor
- Feedback
- Uncertainty

Example: Consider the system in a car where 1 degree throttle change maps to 10 mph change in speed and a 1% change in road grade maps to -5 mph change in speed. How do we relate road grade to throttle change? (Trivially, 1% change equals 0.5 degree throttle)

How might we draw this controller?



The naive open loop controller ignores the disturbance and simply maps the desired speed to 1/10 of that value in throttle so $r = y$ if $w = 0$.



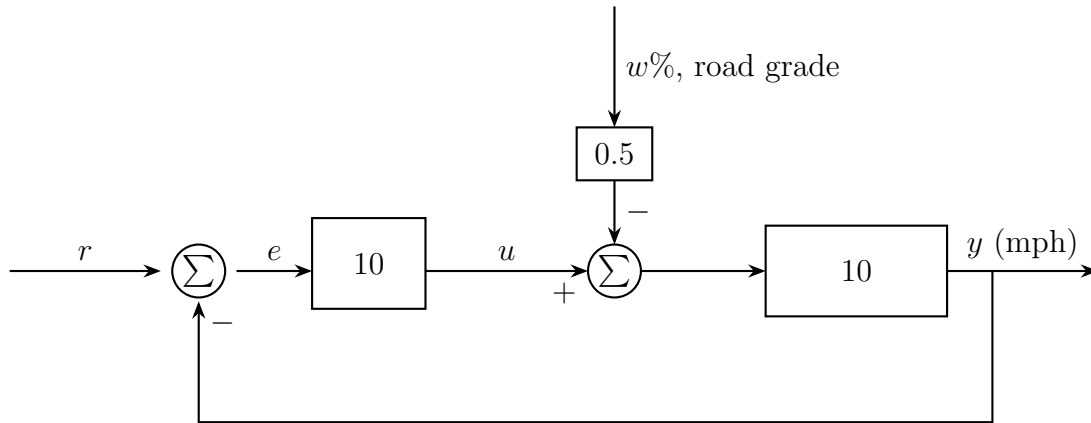
Hence the actual control exerted is

$$\begin{aligned}
 y &= r - 0.5(10)w \\
 &= r - 5w \\
 e &= r - y \\
 &= r - (r - 5w) \\
 &= 5w
 \end{aligned}$$

so our % speed error is $500w/r$. At $r = 65$ mph and $w = 1\%$, we have a 7.7% error in speed.

Remark: in this example we had to choose $1/10$ as the gain of the controller to achieve $r = y$ when $w = 0$. This required knowledge of the plant dynamics. In general, we may not know the plant dynamics and hence cannot choose a controller gain to achieve this. This motivates the use of feedback control.

Let's try this again with feedback control.



If we arbitrarily set the gain at 10, our system is governed by

$$\begin{cases} y = 10(u - 0.5w) \\ u = 10(r - y) \end{cases} \implies y = 100r - 100y - 5w \implies y = \frac{100}{101}r - \frac{5}{101}w \approx 0.99r - 0.05w$$

If $w = 1$ and $r = 65$,

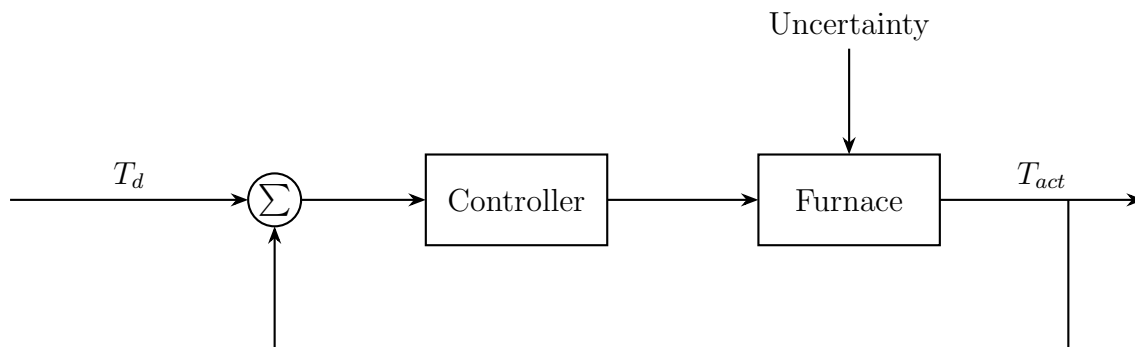
$$e = y - r = \frac{r}{101} + \frac{5}{101}w$$

and the error percentage is

$$e\% = \frac{100e}{r} = \frac{0.01r - 0.05}{r} \approx 0.076\%$$

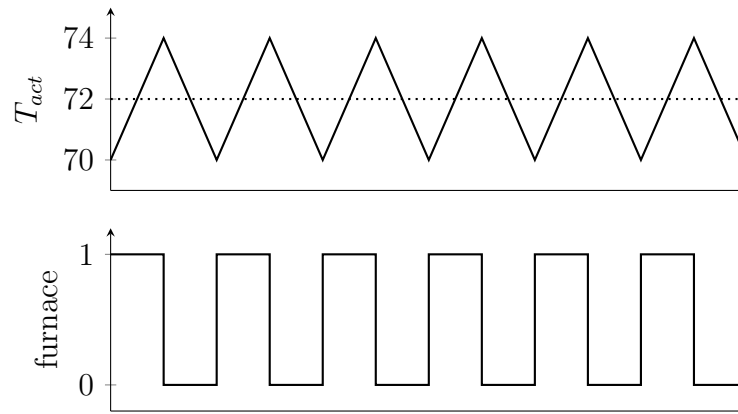
What if we set the gain to 1000? Wouldn't that send the error to 0.0007%? Not necessarily. This can lead to instability and oscillations.

Example: Consider a house with a heater and a thermostat.



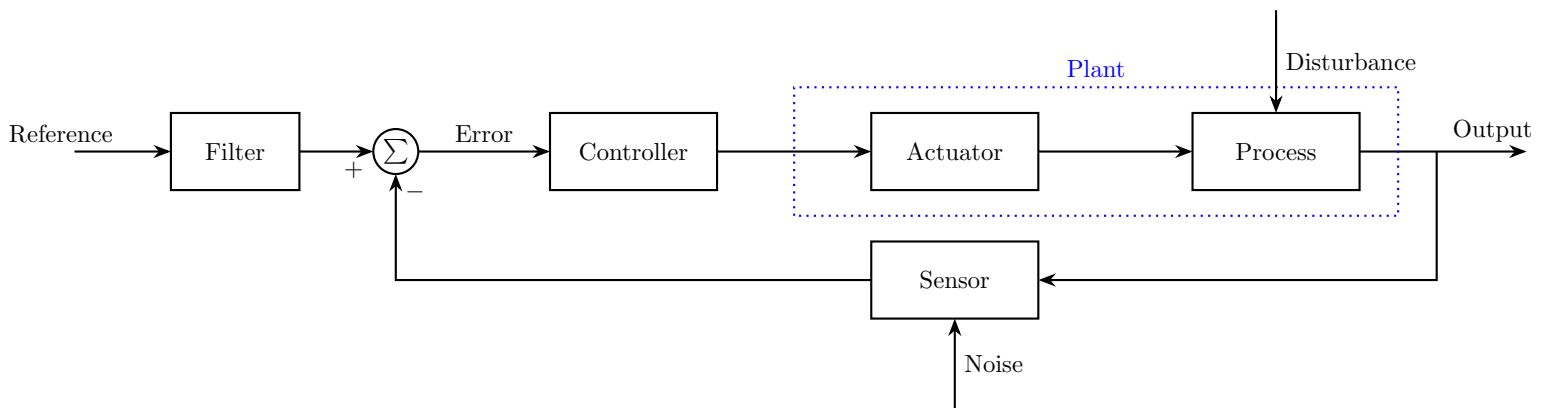
$$e = T_d - T_{act}$$

Notice here the furnace can only turn on and off so we have a delay in the system response. This results in the actual temperature sawtoothing around the setpoint:



3 Sept 17

Recall: last time we introduced a general block diagram for a *Single-input single-output* (SISO) feedback control system.



Characteristics of a well-designed control system:

- *Stability*: the system should not diverge or oscillate uncontrollably
- *Tracking*: the system should follow the desired reference accurately
- *Disturbance rejection*: the system should be able to maintain performance in the presence of disturbances
- *Robustness*: the system should be able to handle model uncertainties and variations in system parameters

As we will see, each box in the block diagram can be represented by a mathematical model (in fact an ODE), and we can analyze the overall system behavior using these models.

Laplace Transform: for a bounded function $f(t)$, the Laplace transform is defined as

$$\mathcal{L}[f(t)] = F(s) = \int_0^{\infty} f(t) e^{-st} dt$$

where $s = \sigma + j\omega$ is a complex variable.

(*Remark:* we define the integral from 0 to ∞ because we are only interested in causal systems, i.e. systems that do not depend on future inputs.)

Examples:

$$\begin{aligned}\mathcal{L}[u(t)] &= 1/s && (u(t) \text{ unit step function}) \\ \mathcal{L}[tu(t)] &= 1/s^2 \\ \mathcal{L}[t^2u(t)] &= 1/s^3 \\ \mathcal{L}[\sin(\omega t)u(t)] &= \frac{\omega}{s^2 + \omega^2} \\ \mathcal{L}[\cos(\omega t)u(t)] &= \frac{s}{s^2 + \omega^2} \\ \mathcal{L}[\delta(t)] &= 1 && (\delta(t) \text{ Dirac delta function})\end{aligned}$$

In Matlab, we can compute these transforms using the **Symbolic Math Toolbox** and

```
1 syms s t w
2 Laplace(cos(w * t))
```

Properties of the Laplace transform:

1. Linearity

$$\mathcal{L}[af(t) + bg(t)] = aF(s) + bG(s)$$

2. Frequency shift

$$\mathcal{L}[e^{-at}f(t)] = F(s + a)$$

3. Time shift

$$\mathcal{L}[f(t - T_d)] = F(s)e^{-T_d s}$$

4. Scaling

$$\mathcal{L}[f(at)] = \frac{1}{a}F\left(\frac{s}{a}\right)$$

5. Integration/differentiation

$$\begin{aligned}\mathcal{L}\left[\frac{d^n f(t)}{dt^n}\right] &= s^n F(s) + s^{n-1}f(0) + s^{n-2}f'(0) + \dots + f^{(n-1)}(0) \\ \mathcal{L}\left[\int_0^t f(t) dt\right] &= \frac{1}{s}F(s)\end{aligned}$$

6. Behavior under convolution

$$\mathcal{L}[f(t) \star g(t)] = \mathcal{L}\left[\int_0^t f(t - \tau)g(\tau) dt\right] = F(s)G(s)$$

7. Final/initial values

$$\lim_{t \rightarrow \infty} f(t) = \lim_{s \rightarrow 0} sF(s)$$

Proportional-Integral-Derivative Control: For $k_p, k_i, k_d \in \mathbb{R}$.

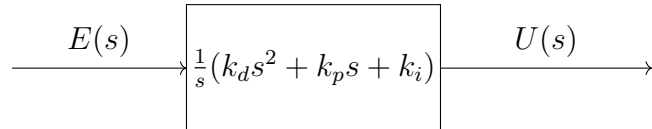
$$u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \frac{de(t)}{dt}$$

$$U(s) = k_p E(s) + k_i \frac{E(s)}{s} + k_d s E(s)$$

Notice by grouping terms,

$$\frac{U(s)}{E(s)} = \frac{k_d s^2 + k_p s + k_i}{s}$$

so by using the Laplace transform, we can represent the PID controller as a polynomial transfer function



Exercise: Suppose there is a mass-on-spring system governed by

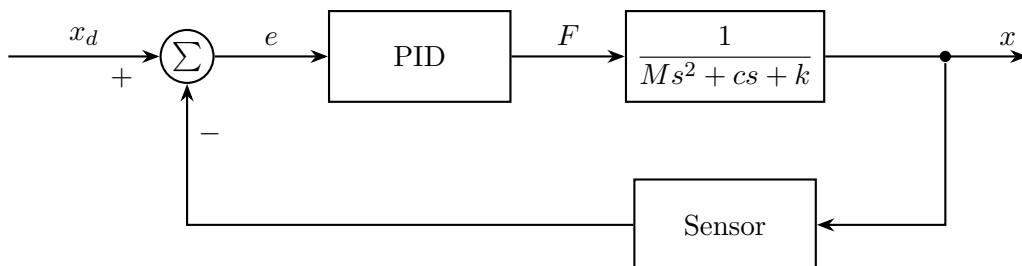
$$F = M\ddot{x} + c\dot{x} + kx$$

What is the transfer function?

$$\begin{aligned} \mathcal{L}[F] &= M(s^2 X(s) - sx(0) - x'(0)) + c(sX(s) - x(0)) + kX(s) \\ &= (Ms^2 + cs + k)X(s) \end{aligned}$$

$$\frac{X(s)}{F(s)} = \frac{1}{Ms^2 + cs + k}$$

This will allow us to control the system by a simple block diagram



4 Sept 22

Example: Use the Laplace transform to solve the following ODE:

$$\begin{cases} \ddot{y} + 5\dot{y} + 6y = 0 \\ y(0) = 2 \\ \dot{y} = 3 \end{cases}$$

$$\begin{aligned}
\mathcal{L}[\ddot{y} + 5\dot{y} + 6y] &= \mathcal{L}[0] \\
s^2Y(0) - sy(0) - \dot{y}(0) + 5(sY(s) - y(0)) + 6Y(s) &= 0 \\
(s^2 + 5s + 6)Y(s) - 2s - 3 - 10 &= 0 \\
Y(s) &= \frac{2s + 13}{s^2 + 5s + 6} \\
y(t) &= \mathcal{L}^{-1} \left[\frac{2s + 13}{s^2 + 5s + 6} \right]
\end{aligned}$$

In this example,

$$Y(s) = \frac{2s + 13}{s^2 + 5s + 6}$$

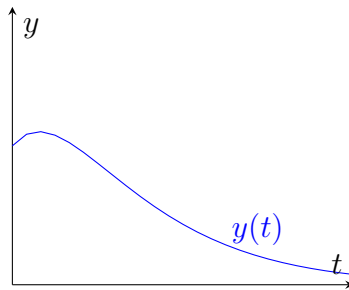
is our **transfer function**. We call the numerator our **zeroes** and the denominator our **poles**.

4.1 Inverse Laplace Transform

We use the **inverse laplace transform** to go from the transfer function back to the time domain.

In this problem, we need to use *partial fractions*:

$$\begin{aligned}
\frac{2s + 13}{(s + 2)(s + 3)} &= \frac{A}{s + 2} + \frac{B}{s + 3} \\
2s + 13 &= A(s + 3) + B(s + 2) \\
2s + 13 &= (A + B)s + (3A + 2B) \\
\begin{cases} A + B &= 2 \\ 3A + 2B &= 13 \end{cases} &\implies A = 9, B = -7 \\
y(t) &= \mathcal{L}^{-1} \left[\frac{9}{s + 2} - \frac{7}{s + 3} \right] \\
&= \boxed{9e^{-2t} - 7e^{-3t}}
\end{aligned}$$



Example:

$$F(s) = \frac{1}{s(s + 0.1)^2} = \frac{K_1}{s} + \frac{K_2}{(s + 0.1)^2} + \frac{K_3}{(s + 0.1)}$$

Choosing K_1 and K_2 to cancel the denominators, we have

$$\begin{aligned}
K_1 &= \frac{1}{(0.1)^2} = 100 \\
K_2 &= \frac{1}{s} \Big|_{s=-0.1} = -10
\end{aligned}$$

and then for K_3 we just need to solve for the remaining constant. For ease of calculation, let $s = 1$:

$$\begin{aligned}\frac{1}{s(s+0.1)^2}\bigg|_{s=1} &= \left[\frac{K_1}{s} + \frac{K_2}{(s+0.1)^2} + \frac{K_3}{(s+0.1)} \right]_{s=1} \\ \frac{1}{1(1.1)^2} &= 100 + \frac{-10}{(1.1)^2} + \frac{K_3}{1.1} \\ K_3 &= \frac{1}{1.1} - 100 + \frac{10}{(1.1)^2} \approx -90.74\end{aligned}$$

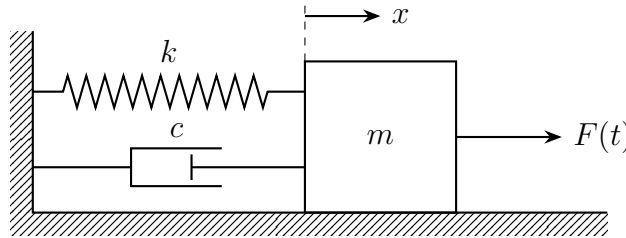
Note that when using partial fractions, we must have a term for each order of the repeated root.

This gives rise to a general transfer function of the form

$$\begin{aligned}H(s) &= \frac{a_n s^n + a_{n-1} s^{n-1} + \dots + a_1}{s^m + b_{m-1} s^{m-1} + \dots + b_1} \\ &= \frac{N(s)}{(s^2 + 2\zeta\omega_n s + \omega_n^2) \prod_{j=3}^n \frac{1}{s-s_j}}\end{aligned}$$

Example: With null initial conditions, the mass-spring-damper system is governed by

$$m\ddot{x} + c\dot{x} + kx = F(t)$$



The transfer function is

$$\frac{X(s)}{F(s)} = \frac{1}{ms^2 + cs + k}$$

We want to represent it as a **second order system** of the form

$$s^2 + 2\zeta\omega_n s + \omega_n^2$$

Notice

$$\frac{1}{ms^2 + cs + k} = \frac{1/m}{s^2 + \frac{c}{m}s + \frac{k}{m}}$$

Now $\omega_n = \sqrt{k/m}$ is the **natural frequency** while $2\zeta\omega_n = c/m$ is the **damping ratio**.

This second order form is essential because it allows us to decompose all higher order systems into a product of second order systems and first order systems.

Now consider the poles (the roots of the denominator $s^2 + 2\zeta\omega_n s + \omega_n^2$). What happens under different values of ζ ?

$$\begin{cases} \zeta = 0 & \implies s_1, s_2 = \pm j\omega_n \\ 0 < \zeta < 1 & \implies s_1, s_2 = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2} \\ \zeta = 1 & \implies s_1, s_2 = -\omega_n \\ \zeta > 1 & \implies s_1, s_2 \in \mathbb{R} : s_2 < \omega_n < s_1 \end{cases}$$

Exercise: Find the inverse Laplace transform of

$$\frac{5}{(s+1)(s^2+4s+8)}$$

First compare

$$s^2 + 4s + 8 = s^2 + 2\zeta\omega_n s + \omega_n^2$$

so $\omega_n = 2\sqrt{2}$ and $\zeta = 1/\sqrt{2}$.

From our analysis above, this means we have two complex conjugate poles at

$$s = -2 \pm j\sqrt{8 - \frac{8}{\sqrt{2}}}$$

Hence to get our inverse laplace, it suffices to write our function in the form

$$\begin{aligned} H(s) &= \frac{A}{s^2 + 2\zeta\omega_n s + \omega_n^2} \\ &= \frac{A}{s^2 + 2\zeta\omega_n s + \omega_n^2 + \zeta^2\omega_n^2 - \zeta^2\omega_n^2} \\ &= \frac{A}{(s^2 + \zeta\omega_n)^2 + \omega_n^2(1 - \zeta^2)} \end{aligned}$$

which has general inverse laplace transform

$$H(s) = \frac{A}{s^2 + 2\zeta\omega_n s + \omega_n^2} \iff h(t) = Ae^{-\zeta\omega_n t} \cos(\omega_n t \sqrt{1 - \zeta^2})$$

In our case,

$$\begin{aligned} H(s) &= \frac{5}{(s+1)(s^2+4s+8)} \\ &= \frac{k_1 s + k_2}{s^2 + 4s + 8} + \frac{k_3}{s + 1} \end{aligned}$$

We can solve for k_1, k_2, k_3 by multiplying both sides by the denominator and choosing convenient values of s .

$$\begin{aligned} 5 &= (k_1 s + k_2)(s + 1) + k_3(s^2 + 4s + 8) \\ s = -1 &\implies 5 = k_3(1 - 4 + 8) \\ &\implies k_3 = 5/5 = 1 \\ s = 0 &\implies 5 = k_2 + 8k_3 \\ &\implies k_2 = -3 \\ s = 1 &\implies 5 = 2(k_1 - 3) + 13 \\ &\implies k_1 = -1 \end{aligned}$$

All together,

$$\begin{aligned} F(s) &= \frac{1}{s+1} - \frac{s+3}{s^2+4s+8} \\ &= \frac{1}{s+1} - \frac{s+2}{(s+2)^2+4} - \frac{1}{2} \frac{2}{(s+2)^2+4} \end{aligned}$$

via completing the square.

From an inverse laplace table, we have

$$\begin{aligned}\mathcal{L}^{-1}\left[\frac{1}{s+1}\right] &= e^{-t} \\ \mathcal{L}^{-1}\left[\frac{s+2}{(s+2)^2+4}\right] &= e^{-2t}\cos(2t) \\ \mathcal{L}^{-1}\left[\frac{2}{(s+2)^2+4}\right] &= e^{-2t}\sin(2t)\end{aligned}$$

and

$$f(t) = e^{-t} + e^{-2t}[\cos(2t) + \sin(2t)]$$

This general method works if the denominator is of higher order than the numerator. If the numerator is of higher order, we can use polynomial long division to reduce the order of the numerator.

Example:

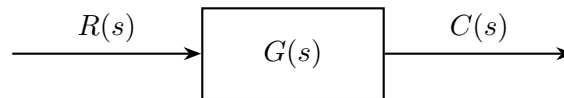
$$\frac{s^2 + 4s + 16}{s^2 + 5s + 25} = 1 - \frac{s + 9}{s^2 + 5s + 25}$$

but this is a problem! $\mathcal{L}^{-1}[1] = \delta(t)$ which means the system is returning an impulse from no input.

4.2 Block Diagrams

We have three primary ways to connect systems together: *series*, *parallel*, and *feedback*.

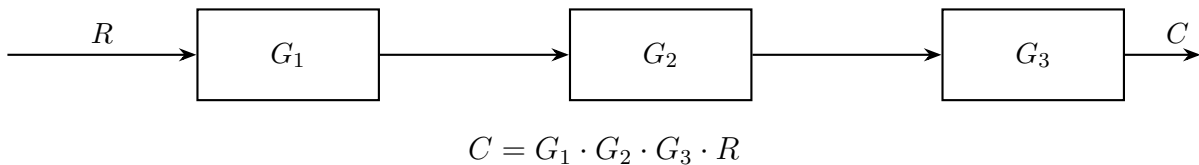
The simplest series block diagram is



which is equivalent to

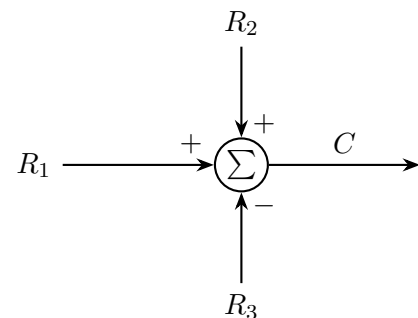
$$C(s) = G(s) \cdot R(s)$$

This could also look like a *cascade*:



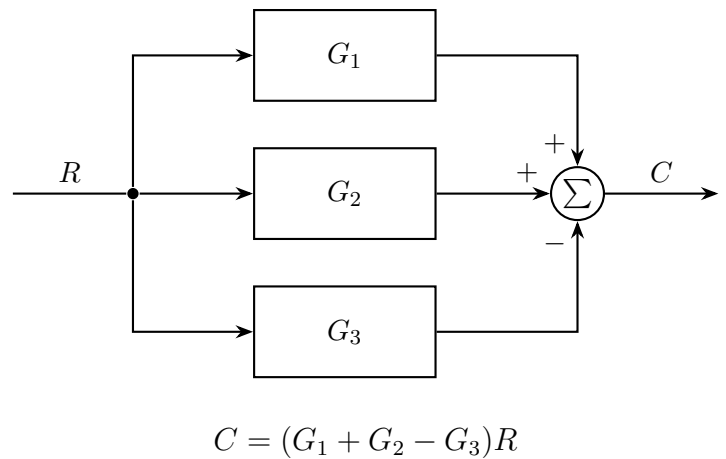
Parallel block diagrams appear in two forms:

Multi-input:

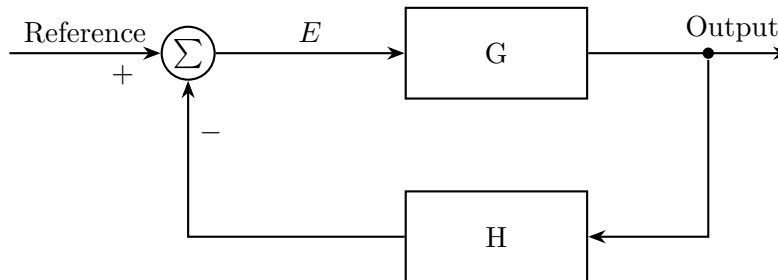


$$C = R_1 + R_2 - R_3$$

Multi-path:



Feedback block diagrams are used to represent systems with feedback loops.

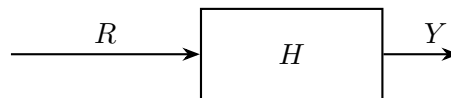


What is the transfer function of this system?

$$\begin{cases} E = R - CH \\ C = GE \end{cases} \implies C = G(R - CH) \implies C = \frac{GR}{1 + GH} \implies \frac{C}{R} = \frac{G}{1 + GH}$$

5 Sept 24

Recall: For a block diagram



the transfer function is

$$H(s) = \frac{Y(s)}{R(s)}$$

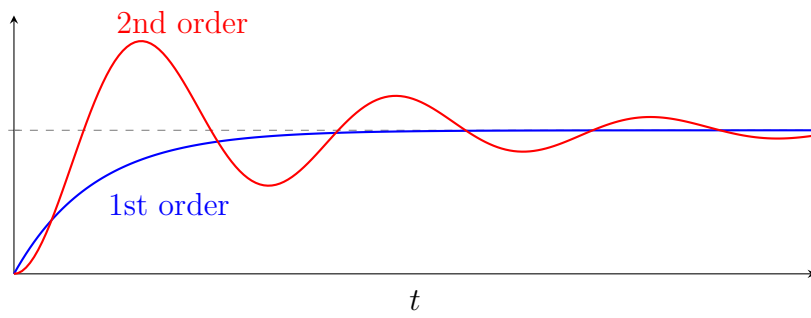
and the transfer function can be of any order:

$$H(s) = \frac{a_0 s^n + a_1 s^{n-1} + \dots + a_{n-1} s + a_n}{b_0 s^m + b_1 s^{m-1} + \dots + b_{m-1} s + b_m}$$

If $m > n$, we can use partial fractions to decompose the transfer function into a sum of first (a system with real poles) and second order systems (those with complex poles).

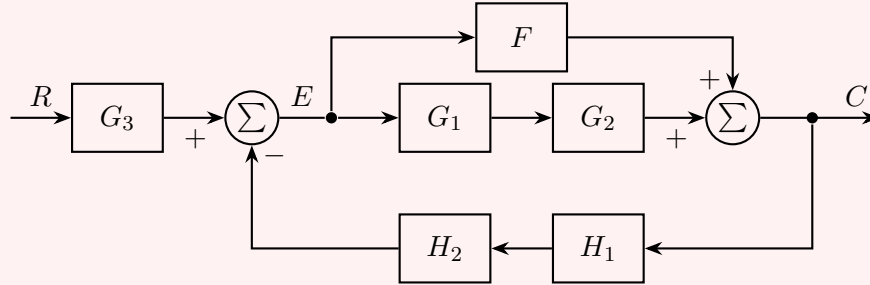
If $m = n$, the transfer function will be of the form $H(s) = 1 + \dots$

In general, a first order system will nicely converge to a steady state value, while a second order system will oscillate around the steady state value.



We can always represent the poles of a second order system in the form $s^2 + 2\zeta\omega_n s + \omega_n^2$ for $0 \leq \zeta \leq 1$. (If $\zeta = 1$, the system will have two real poles).

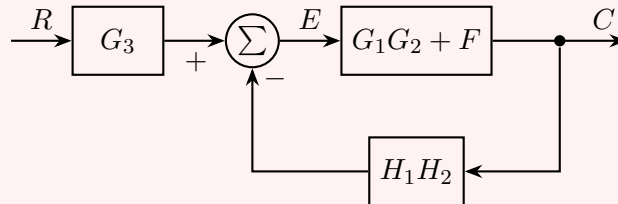
Exercise: What is the transfer function of the following system?



$$\begin{cases} E = RG_3 - H_2H_1C \\ C = EF + EG_1G_2 \\ \quad = E(F + G_1G_2) \end{cases}$$

$$\begin{aligned} C &= (RG_3 - H_2H_1C)(F + G_1G_2) \\ &= RG_3(F + G_1G_2) - H_2H_1C(F + G_1G_2) \\ C[1 + H_2H_1(F + G_1G_2)] &= RG_3(F + G_1G_2) \\ \frac{C}{R} &= \frac{G_3(F + G_1G_2)}{1 + H_2H_1(F + G_1G_2)} \end{aligned}$$

We can also use block diagram reduction to simplify the system. First, we can combine G_1 and G_2 in series, then combine H_1 and H_2 in series, then combine the two feedback loops into one, and finally combine G_3 with the rest of the system in series.

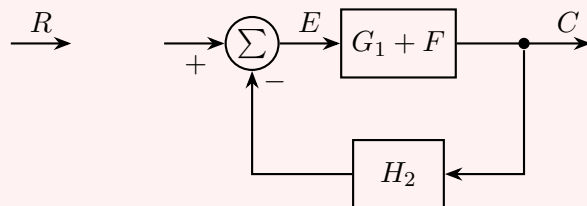


Which, using the general formula, gives

$$\frac{C}{G_3R} = \frac{F + G_1G_2}{1 + (H_1H_2)(F + G_1G_2)} \implies \frac{C}{R} = \frac{G_3(F + G_1G_2)}{1 + (H_1H_2)(F + G_1G_2)}$$

In the example above, we call the F branch a *feedforward* branch, while the H_1H_2 branch is a *feedback* branch. G_1G_2 is assumed to be the *plant control*.

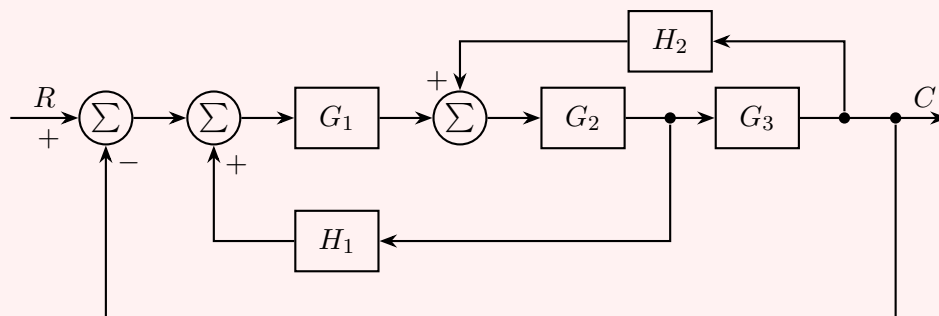
Exercise: Find the transfer function of the system



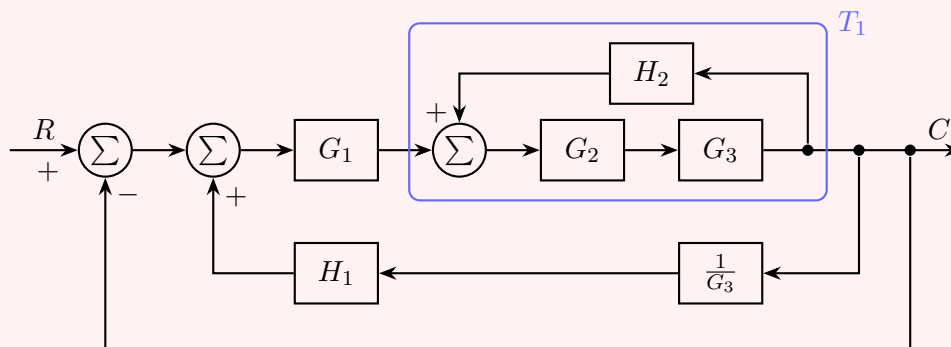
where $F = 1$, $G_1 = \frac{5}{s+5}$, $H_2 = \frac{7}{s+7}$,

$$\begin{aligned} \frac{C}{R} &= \frac{G_1 + F}{1 + H_2(G_1 + F)} \\ &= \frac{\frac{5}{s+5} + 1}{1 + \frac{7}{s+7} \left(\frac{5}{s+5} + 1 \right)} \\ &= \frac{\frac{5+s+5}{s+5}}{\frac{(s+7)(s+5) + 7(5+s+5)}{(s+7)(s+5)}} \\ &= \frac{(s+7)(s+10)}{(s+7)(s+5) + 7(s+10)} \\ &= \frac{(s+7)(s+10)}{s^2 + 19s + 105} \end{aligned}$$

Exercise: Find the transfer function of the system



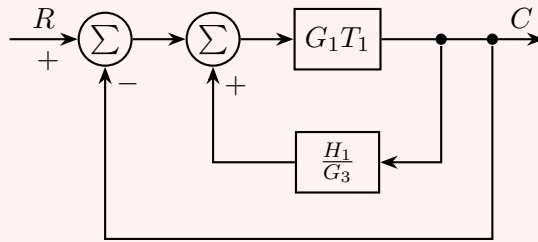
The trick is to simplify:



Inner block:

$$T_1 = \frac{G_2 G_3}{1 - G_2 G_3 H_2}$$

(notice the flipped sign in the denominator because the feedback is positive).

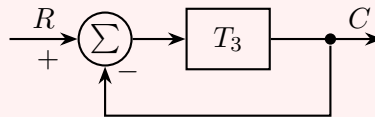


Then letting,

$$T_2 = \frac{H_1}{G_3}$$

we have

$$T_3 = \frac{G_1 T_1}{1 - G_1 T_1 T_2}$$



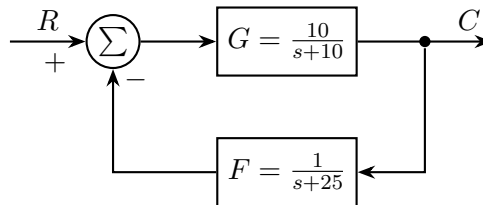
And finally,

$$\frac{C}{R} = \frac{T_3}{1 + T_3}$$

As you can see, block diagram reduction is a very useful tool for simplifying complex systems. It allows us to break down the system into smaller, more manageable parts, and then combine them to find the overall transfer function. However, this is still a very tedious process, and it is easy to make mistakes.

Let's use Matlab instead.

Example:



```

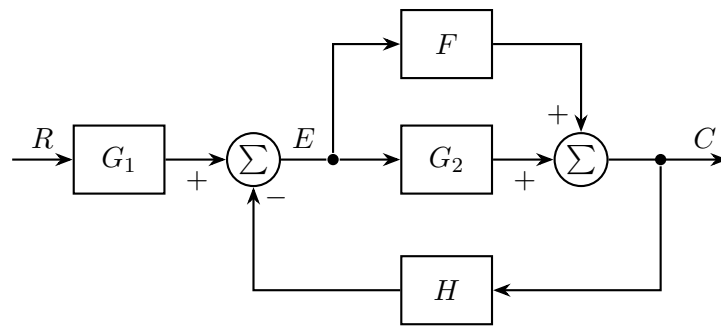
1  s = tf('s');
2  G = 10/(s + 10);
3
4  step(G) % gets the step response of the system
5  impulse(G) % gets the impulse response of the system
6
7  F = 1/(s + 25);
8  CLTF = feedback(G, F) % gets the closed loop transfer function of the system

```

This will output

$$\text{CLTF} = \frac{10s + 250}{s^2 + 35s + 250}$$

Example:



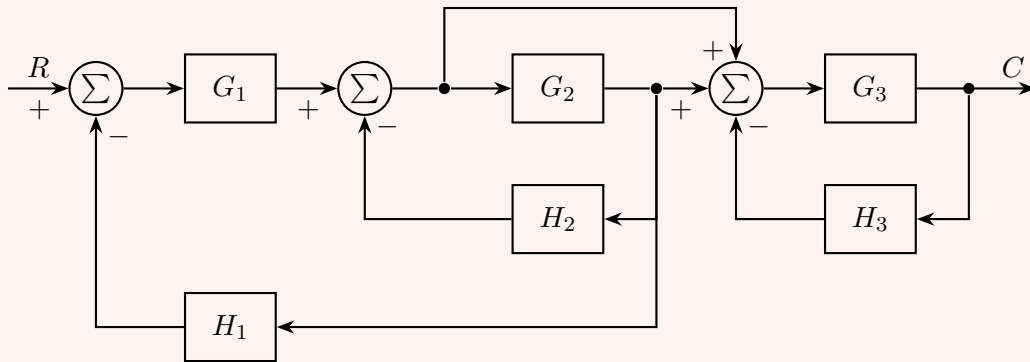
We can define $Q_1 = F + G_2$ so the system can be solved with

```

1  s = tf('s');
2  syms F G1 G2 H;
3  Q = G + G2;
4
5  M = feedback(Q, H);
6  CLTF = Series(M, G1)
7
8  ltview(CLTF) % opens the linear system analyzer

```

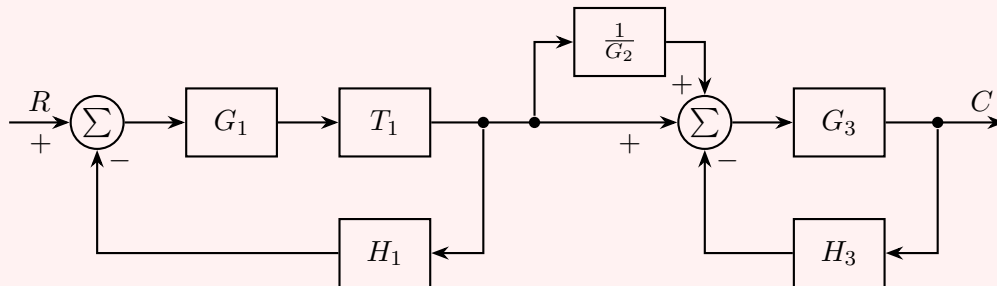
Exercise: Find $\frac{C}{R}$ for the system



Letting

$$T_1 = \frac{G_2}{1 + G_2 H_2}$$

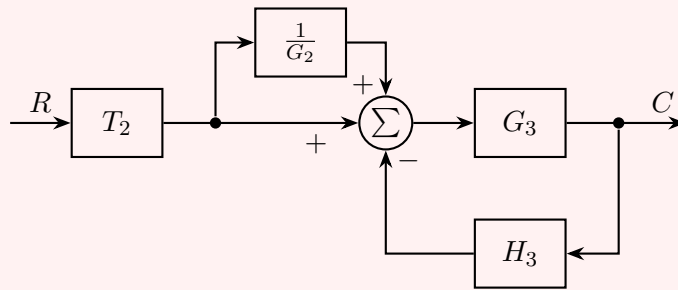
and moving the feedforward branch to after T_1 , we have



With

$$T_2 = \frac{T_1 G_1}{1 + T_1 G_1 H_1}$$

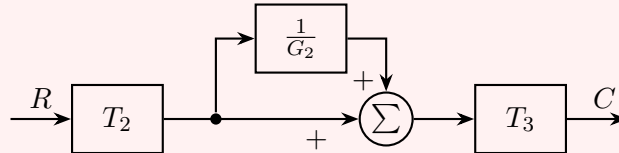
the first section simplifies into a single block:



And finally,

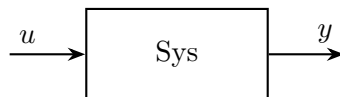
$$T_3 = \frac{G_3}{1 + G_3 H_3}$$

gives a simple parallel-series system to be solved by matlab:



6 Sept 29

We can view a system

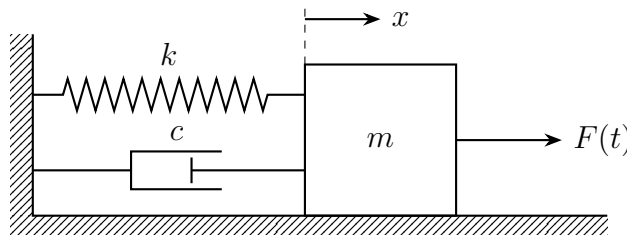


in three ways:

1. Differential Equations (e.g. $\ddot{y} + 2t\dot{y} + y = u$ which cannot be solved via Laplace because it is not LTI)
2. Transfer function (e.g. $\frac{Y}{U} = \frac{s+1}{s^2+4s+11}$)
3. State space (e.g. $\dot{X} = Ax + bu$, $Y = cx + du$)

As it happens, during the Space Race, the Americans were using transfer functions which excelled at single-input, single-output systems. For Sputnik, Lyapunov developed the state-space formulation which can handle *multi-input, multi-output systems*.

Example (converting to state space): Recall the spring-mass-damper system.



$$m\ddot{x} + c\dot{x} + kx = f$$

We want to write this system as a state-space system of the form

$$\begin{cases} \dot{X} = Ax + bu \\ Y = cx + du \end{cases}$$

In general, the order of the differential equation will dictate the size of the matrix A .

For a second-order ODE, $A \in \mathbb{R}^{2 \times 2}$, $u \in \mathbb{R}^{2 \times 1}$, $c \in \mathbb{R}^{1 \times 2}$, $d \in \mathbb{R}^{1 \times 1}$, $x \in \mathbb{R}^{2 \times 1}$.

Define a **state variable**

$$X = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

and suppose $x_1 = x$ (the position of the mass) and $x_2 = \dot{x}$ (the velocity).

In particular, this means

$$\dot{x}_1 = x_2$$

so

$$\ddot{x} = \frac{1}{m}(f - c\dot{x} - kx) \implies \dot{x}_2 = \frac{1}{m}(f - cx_2 - kx_1)$$

hence

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \overbrace{\begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}}^A \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \overbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}^B f$$

$$Y = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_C \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

(making the arbitrary choice that we want the position as our output)

If instead we wanted $[x, \dot{x}]$ as our output, we could define

$$Y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

This method lets us move from DiffEq to state space. How do we move from state space to transfer function or transfer function to state space?

Trivially, to move from transfer function to state space just cross-multiply to recover the ODE and then move the state space.

The more difficult direction is state space to transfer function. Just take the Laplace and solve for Y/U .

First, solving for $X(s)$:

$$\begin{aligned} sX(s) &= AX(s) + BU(s) \\ (sI - A)X(s) &= BU(s) \\ X(s) &= (sI - A)^{-1}BU(s) \end{aligned}$$

Then, substituting,

$$\begin{aligned} Y(s) &= CX(s) + DU(s) \\ &= (C(sI - A)^{-1}B + D)U(s) \end{aligned}$$

which gives the general form

$$\boxed{\frac{Y(s)}{U(s)} = C(sI - A)^{-1}B + D}$$

In our example,

$$\begin{aligned}
 (sI - A) &= \begin{bmatrix} s & -1 \\ \frac{k}{m} & s + \frac{c}{m} \end{bmatrix} \\
 (sI - A)^{-1} &= \frac{1}{s(s + \frac{c}{m}) + \frac{k}{m}} \begin{bmatrix} s + \frac{c}{m} & 1 \\ -\frac{k}{m} & s \end{bmatrix} \\
 \frac{Y(s)}{U(s)} &= C(sI - A)^{-1}B + D \\
 &= \frac{1}{s^2 + \frac{cs}{m} + \frac{k}{m}} \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} s + \frac{c}{m} & 1 \\ -\frac{k}{m} & s \end{bmatrix} \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} \\
 &= \frac{1}{s^2 + \frac{cs}{m} + \frac{k}{m}} \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{m} \\ \frac{s}{m} \end{bmatrix} \\
 &= \frac{1}{ms^2 + cs + k}
 \end{aligned}$$

Which is exactly what we found via Laplace:

$$m\ddot{x} + c\dot{x} + kx = f \implies ms^2X(s) + csX(s) + kX(s) = F \implies \frac{X(s)}{F(s)} = \frac{1}{ms^2 + cs + k}$$

The general **non-linear state space form** is

$$\begin{cases} \dot{x} = f(x, u) \\ y = h(x, u) \end{cases}$$

Nonlinear systems can be controlled either via **linearization** or **non-linear control**. Only linearization will be treated in this course.

Two basic methods: make a linear approximation (e.g. $\sin \theta = \theta$) or linearize around equilibria.

Example: Consider a transatlantic flight. Over the length of the flight, fuel usage can lead to a change in mass of almost 40%. We would like our controller to respond to this change without needing to model the system continuously.

We can take advantage of times when the plane has constant speed (e.g. no acceleration) to solve for equilibria

$$\begin{cases} \dot{x} = 0 \\ f(x, u)|_{x_0, u_0} = 0 \\ y = h(x_0, y_0) \end{cases} \implies \begin{matrix} A = \frac{\partial f}{\partial x}|_{u_0, x_0} & C = \frac{\partial h}{\partial x}|_{u_0, x_0} \\ B = \frac{\partial f}{\partial u}|_{u_0, x_0} & D = \frac{\partial h}{\partial u}|_{u_0, x_0} \end{matrix}$$

in a higher dimensional analogue to

$$f(x) = f(x_0) + \frac{\partial f}{\partial x} \Big|_{x_0} (x - x_0)$$

7 Oct 1

State Space Linearization: Consider a nonlinear system

$$\begin{cases} \dot{x} = f(x, u) \\ y = h(x, u) \end{cases}$$

General procedure:

1. Find an equilibrium point (x_0, u_0) such that $f(x_0, u_0) = 0$.
2. Calculate the Jacobians

$$\begin{aligned} A &= f_x(x_0, u_0) \\ B &= f_u(x_0, u_0) \\ C &= h_x(x_0, u_0) \\ D &= h_u(x_0, u_0) \end{aligned}$$

3. Form the state space system

$$\begin{cases} \dot{\bar{x}} = A\bar{x} + B\bar{u} \\ \bar{y} = C\bar{x} + D\bar{u} \end{cases}$$

where $\bar{x} \equiv x - x_0$ and $\bar{u} = u - u_0$.

Example: Linearize the following system about $u_0 = 1$.

$$\begin{cases} \dot{x}_1 = x_1 u - 1 = f_1(x, u) \\ \dot{x}_2 = -x_1^2 + x_2 + u = f_2(x, u) \\ y = x_1 + x_2 = h(x, u) \end{cases}$$

First find the equilibrium point

$$\begin{aligned} \dot{x}_1 = 0 &= x_1 - 1 \implies x_1 = 1 \\ \dot{x}_2 = 0 &= -1 + x_2 + 1 \implies x_2 = 0 \\ y &= x_1 + x_2 = 1 \end{aligned}$$

giving $(x_0 = (x_{01}, x_{02}), u_0) = ((1, 0), 1)$.

Now the Jacobian matrices are

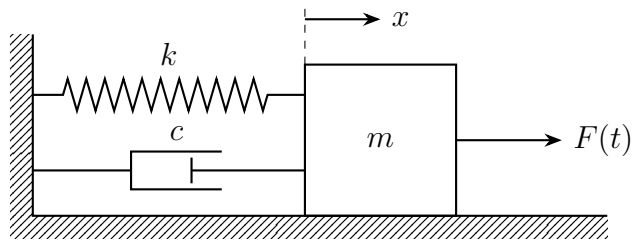
$$\begin{aligned} A &= \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix}_{(x_0, u_0)} = \begin{bmatrix} u & 0 \\ -2x_1 & 1 \end{bmatrix}_{(x_0, u_0)} = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \\ B &= \begin{bmatrix} \frac{\partial f_1}{\partial u} \\ \frac{\partial f_2}{\partial u} \end{bmatrix}_{(x_0, u_0)} = \begin{bmatrix} x_1 \\ 1 \end{bmatrix}_{(x_0, u_0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ C &= \begin{bmatrix} \frac{\partial h}{\partial x_1} & \frac{\partial h}{\partial x_2} \end{bmatrix}_{(x_0, u_0)} = \begin{bmatrix} 1 & 1 \end{bmatrix}_{(x_0, u_0)} = \begin{bmatrix} 1 & 1 \end{bmatrix} \\ D &= \frac{\partial h}{\partial u}_{(x_0, u_0)} = 0 \end{aligned}$$

So our state space system is

$$\begin{aligned} \dot{\bar{x}} &= \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \bar{x} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \bar{u} \\ &= \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} x_1 - 1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} (u - 1) \\ \bar{y} &= \begin{bmatrix} 1 & 1 \end{bmatrix} \bar{x} \\ &= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 - 1 \\ x_2 \end{bmatrix} \end{aligned}$$

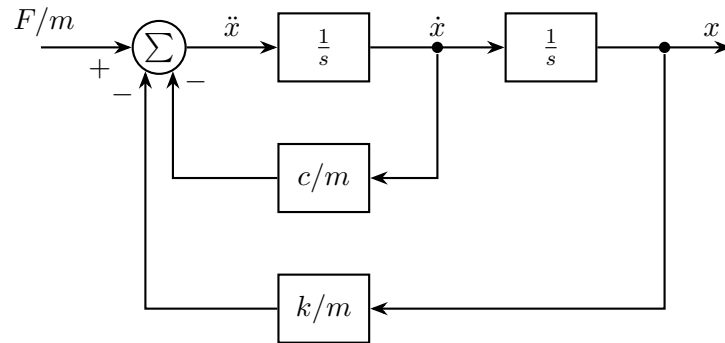
which is a simple linear system that can be solved with the tools we have learned so far though it is only valid for small perturbations around the equilibrium point.

Example (Simulink):

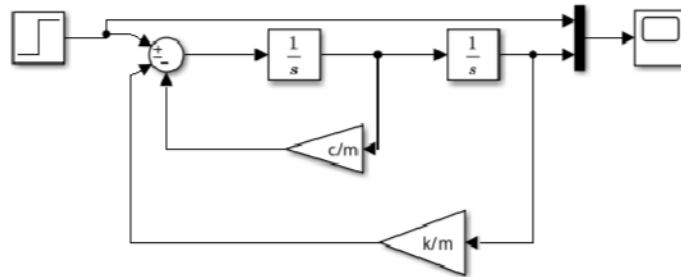


$$m\ddot{x} + c\dot{x} + kx = F$$

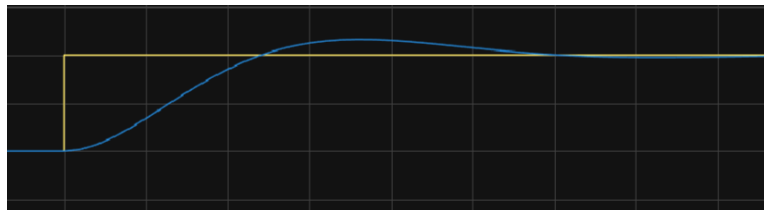
Recalling our Laplace rule for integration, we can draw a block diagram for this system:



Defining c, k, m as variables in our workspace, we can simulate this system in Simulink



Using $m = 1$, $c = 1$, and $k = 1$, we can simulate the system with a step input of $F = 1$ and get the following output:



In the matlab command window, we can also use some very powerful commands:

```

1  % define the system
2  s = tf('s');
3  Y = (s+1)/(s^2 + 5*s + 25);
4
5  % convert to state space
6  Yss = ss(Y)
7
8  % convert back to transfer function
9  Yp = tf(Yss)
10
11 % graph the step response
12 step(Yp)

```

13
14
15
16

```
grid on

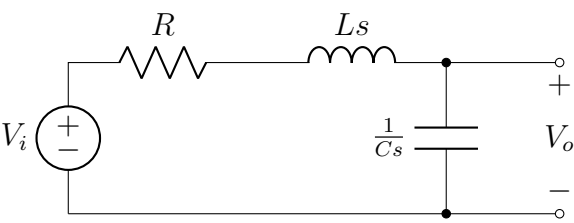
% natural frequency and damping ratio
[omega, zeta] = damp(Yp)
```

8 Oct 6

Modelling Electrical Components:

Component	Symbol	Voltage	Current	Imepedance
Resistor		$V = iR$	$i = \frac{V}{R}$	$Z_R = R$
Capacitor (uF)		$V = \frac{1}{C} \int i dt$	$i = C \frac{dV}{dt}$	$Z_C = \frac{1}{sC}$
Inductor (mH)		$V = L \frac{di}{dt}$	$i = \frac{1}{L} \int V dt$	$Z_L = sL$

Example: What is the transfer function of the following circuit?



$$V_i = iR + iLs + i \frac{1}{Cs} = i(R + Ls + \frac{1}{Cs})$$
$$I(s) = \frac{V_i(s)}{R + Ls + \frac{1}{Cs}}$$
$$V_o = I \frac{1}{Cs}$$
$$\frac{V_o(s)}{V_i(s)} = \frac{\frac{1}{Cs}}{R + Ls + \frac{1}{Cs}} = \frac{1}{LCs^2 + RCs + 1}$$

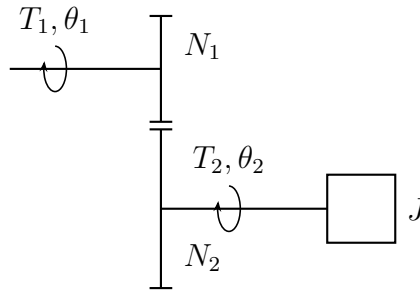
Putting this in the second order form

$$\frac{1}{s^2 + 2\zeta\omega_n s + \omega_n^2} \implies \omega_n^2 = \frac{1}{LC}, \quad 2\zeta\omega_n = \frac{R}{L}$$

Modelling Mechanical Systems:

Component	Symbol	Force/Torque
Mass		$F = m\ddot{x}$
Damper		$F = c\dot{x}$
Spring		$F = kx$
Torsional mass		$T = I\ddot{\theta}$
Torsional damper		$T = c\dot{\theta}$
Torsional spring		$T = k\theta$

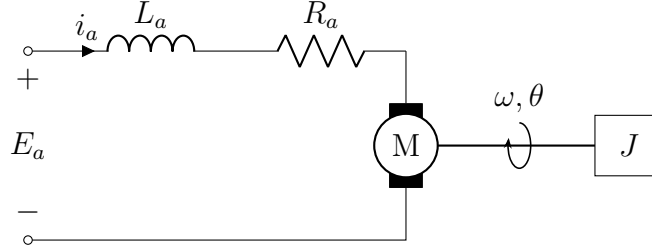
Gears:



For N_i the number of teeth,

$$\frac{T_1}{T_2} = \frac{\theta_2}{\theta_1} = \frac{N_1}{N_2}$$

DC Motor:



where a is the *armature* (related to the winding). We want θ/E and ω/E .

When the motor is spinning, it generates a back EMF $V_{BEMF} = K_b\omega$, and a torque $T = K_t i_a$ where K_b and K_t are constants given in the motor datasheet.

Hence, the system can be described by:

$$E_a = L \frac{di_a}{dt} + R i_a + V_{BEMF}$$

$$T = J \ddot{\theta} = K_T i_a$$

Taking the Laplace transforms,

$$I_a(s) = \frac{J s^2}{K_T} \theta(s)$$

$$E_a(s) = L s I_a(s) + R I_a(s) + K_b s \theta(s)$$

$$= (L s + R) \frac{J s^2}{K_T} \theta(s) + K_b s \theta(s)$$

$$\frac{\theta(s)}{E_a(s)} = \frac{K_T}{J L s^3 + J R s^2 + K_b K_T s}$$

$$\frac{\omega(s)}{E_a(s)} = \frac{K_T}{J L s^2 + J R s + K_b K_T}$$

since $\omega = \dot{\theta} \implies \omega(s) = s \theta(s)$.

In a typical system, we might have $L \approx 10 \text{ mH}$ and $R \approx 20 \Omega$ so $L/R \approx 10^{-4}$, making

$$\frac{\omega}{E_a} \approx \frac{B}{s + A}$$

a reasonable approximation.

Transfer function decomposition: Consider the general transfer function

$$H(s) = \frac{b_1 s^m + b_2 s^{m-1} + \dots + b_{m+1}}{s^n + a_1 s^{n-1} + \dots + a_n} = \frac{k \prod_{i=1}^m (s - z_i)}{\prod_{i=1}^n (s - p_i)}$$

which can be decomposed into products of poles and zeros.

Example:

$$G(s) = \frac{k(s+1)(s+2)}{s(s+3)(s+10)(s^2+s+2)}$$

has zeroes at $\{-1, -2\}$ and poles at $\{0, -3, -10, -1 \pm i\}$.

In matlab, we can use

```
1 s = tf('s');
2 G = (s+1)*(s+2)/(s*(s+3)*(s+10)*(s^2 + s + 2));
3 [omega, zeta, poles] = damp(G)
```

For a first-order system,

$$G(s) = \frac{a}{s+a} = \frac{1}{\tau s + 1}$$

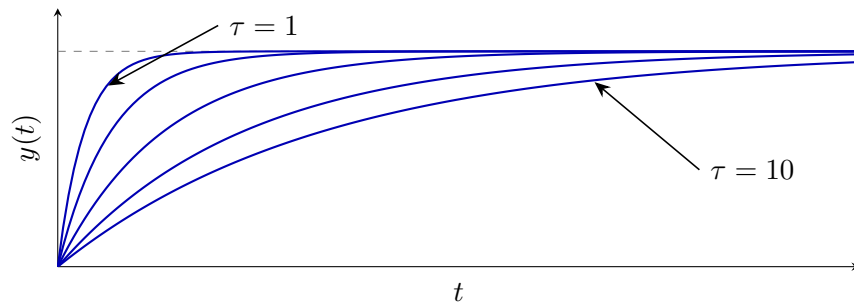
where $\tau = 1/a$ is the time constant.

If

$$Y(s) = \frac{a}{(s+a)} \cdot \frac{1}{s} \Rightarrow y(t) = 1 - e^{-at}$$

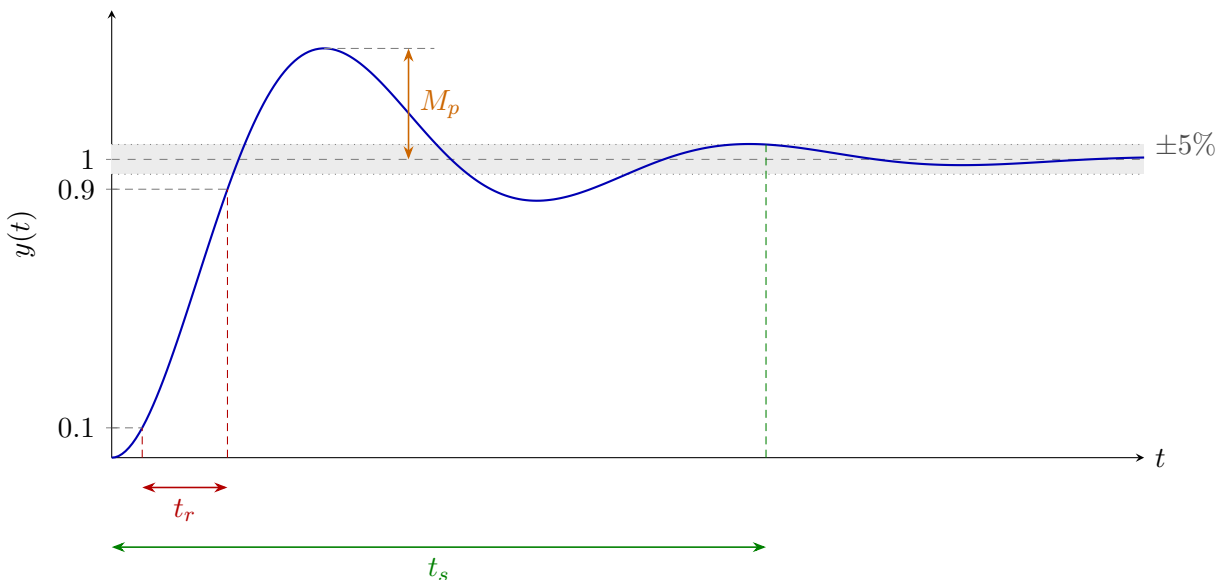
(or else $1 - e^{-t/\tau}$)

As a function of τ , the system response to a step varies:



To specify a first-order system, we can use

- Rise time t_r (time to go from 10% to 90% of final value)
- Settling time t_s (time to reach and stay within 5% of final value)
- Overshoot M_p ($\frac{y_{max}}{y_{ss}} - y_{ss} \cdot 100\%$, though this is zero for all first-order systems)



Example: If $G(s) = \frac{1}{0.1s+1}$, and $T_r = 2.2\tau$ and $T_s = 3\tau$, how long are the rise time and settling time?

$$T_r = 2.2\tau = 2.2(0.1) = 0.22$$

$$T_s = 3\tau = 3(0.1) = 0.3$$

The canonical form for a second-order system is

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

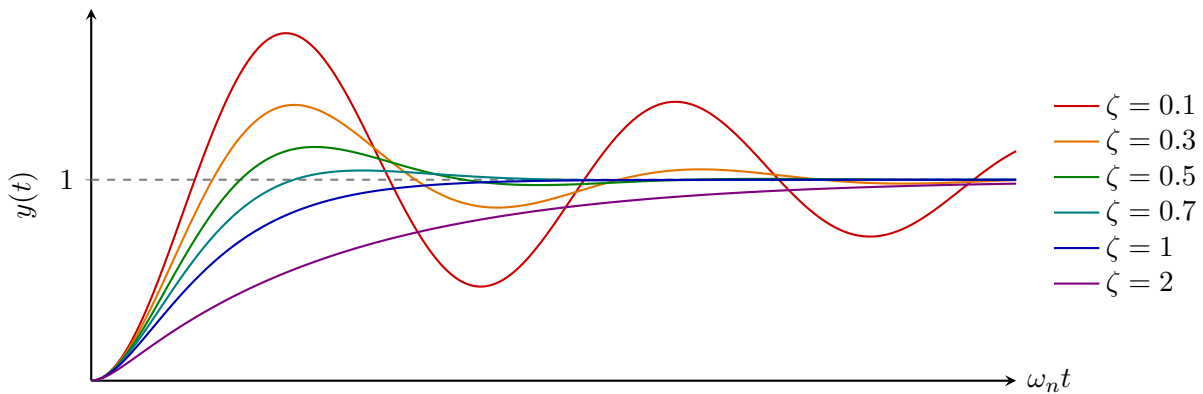
which responds to a step input $1/s$ as

$$Y(s) = \frac{\omega_n^2}{s(s^2 + 2\zeta\omega_n s + \omega_n^2)}$$

$$y(t) = 1 - e^{-\zeta\omega_n t} \left(\cos(\omega_d t) + \frac{\zeta}{\sqrt{1-\zeta^2}} \sin(\omega_d t) \right)$$

where $\omega_d = \omega_n \sqrt{1-\zeta^2}$ is the damped natural frequency.

For fixed ω_n , decreasing ζ gives a faster rise but more overshoot and oscillation:



9 Oct 8

Recall: A first order system has canonical transfer function

$$G(s) = \frac{C(s)}{R(s)} = \frac{1}{\tau s + 1} = \frac{a}{s + a}, \quad a = 1/\tau$$

corresponding to the differential equation

$$\dot{c} + ac = ar$$

For a unit step input,

$$C(s) = G(s)R(s) = \frac{a}{s+a} \cdot \frac{1}{s}$$

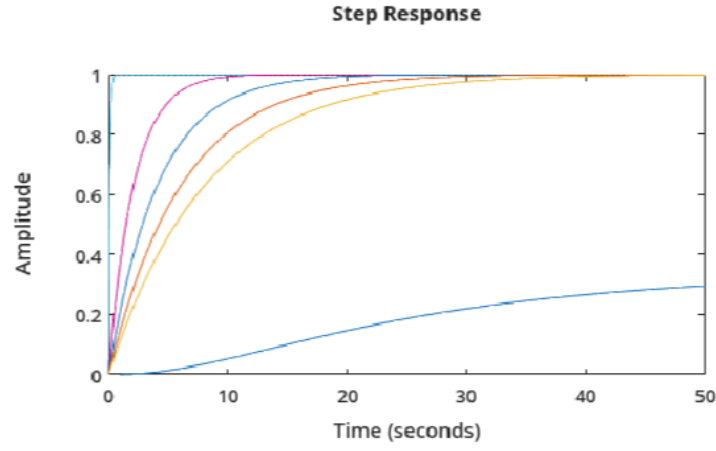
$$c(t) = 1 - e^{-at} = 1 - e^{-t/\tau}$$

```

1 figure(1)
2 for tau=0.1:2:10
3     Y = tf(1, [tau 1]);
4     hold on;
5     step(Y);
6 end

```

produces



The *rise time* is the time it takes to go from 10% to 90% of the final value, and the *settling time* is the time it takes to reach and stay within a specified percentage of the final value (commonly 5%).

A second order system has canonical transfer function

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} = \frac{\omega_n^2}{(s + \zeta\omega_n)^2 + \omega_n^2(1 - \zeta^2)}$$

corresponding to the differential equation

$$\ddot{c} + 2\zeta\omega_n\dot{c} + \omega_n^2 c = \omega_n^2 r$$

where ω_n is the undamped natural frequency (rad/s), ζ is the damping ration, and ω_d is the damped natural frequency (rad/s).

Under a step response,

$$\begin{aligned} C(s) &= G(s)R(s) \\ &= \frac{\omega_n^2}{s(s^2 + 2\zeta\omega_n s + \omega_n^2)} \\ &= \frac{1}{s} + \frac{A_1 s + A_2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \\ &= \frac{1}{s} - \frac{s + \zeta\omega_n + \omega_d(\zeta\omega_n/\omega_d)}{(s + \zeta\omega_n)^2 + \omega_d^2} \\ &= \frac{1}{s} - \frac{s + \zeta\omega_n + \omega_d(\zeta/\sqrt{1 - \zeta^2})}{(s + \zeta\omega_n)^2 + \omega_d^2} \\ c(t) &= 1 - e^{-\zeta\omega_n t} \left(\cos(\omega_d t) + \frac{\zeta}{\sqrt{1 - \zeta^2}} \sin(\omega_d t) \right) \\ &= 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1 - \zeta^2}} \left[\sqrt{1 - \zeta^2} \cos(\omega_d t) + \zeta(\omega_d t) \right] \\ &= 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1 - \zeta^2}} \cos(\omega_d t - \phi), \quad \phi = \sin^{-1}(\zeta) \end{aligned}$$

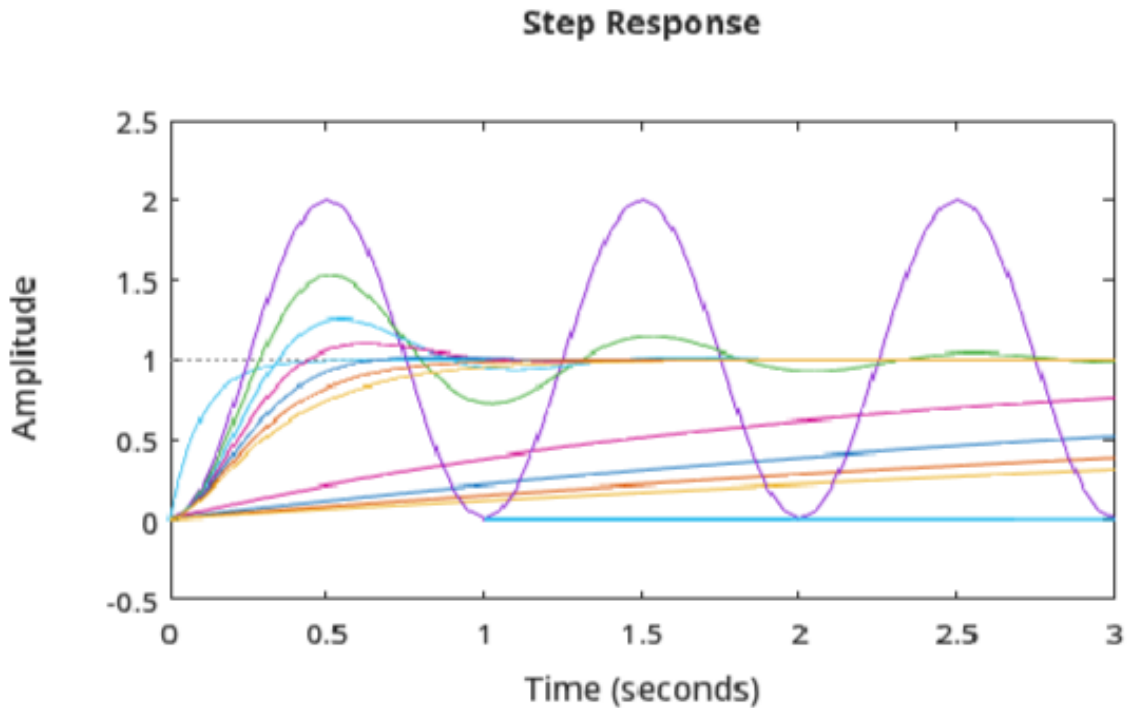
if $0 < \zeta < 1$ (underdamped) then we have two complex conjugate poles, if $\zeta = 1$ (critically damped) then we have a repeated real pole, and if $\zeta > 1$ (overdamped) then we have two distinct real poles. If $\zeta < 0$, we have an unstable system and the step response will diverge. If $\zeta = 0$, the system is undamped and the step response will oscillate indefinitely.

These particular equations, however, will collapse for $\zeta \geq 1$ so we need to go back to the inverse Laplace transform to find the step response for these cases.

Note that Matlab does not care about $\zeta \geq 1$ since it uses the state space form.

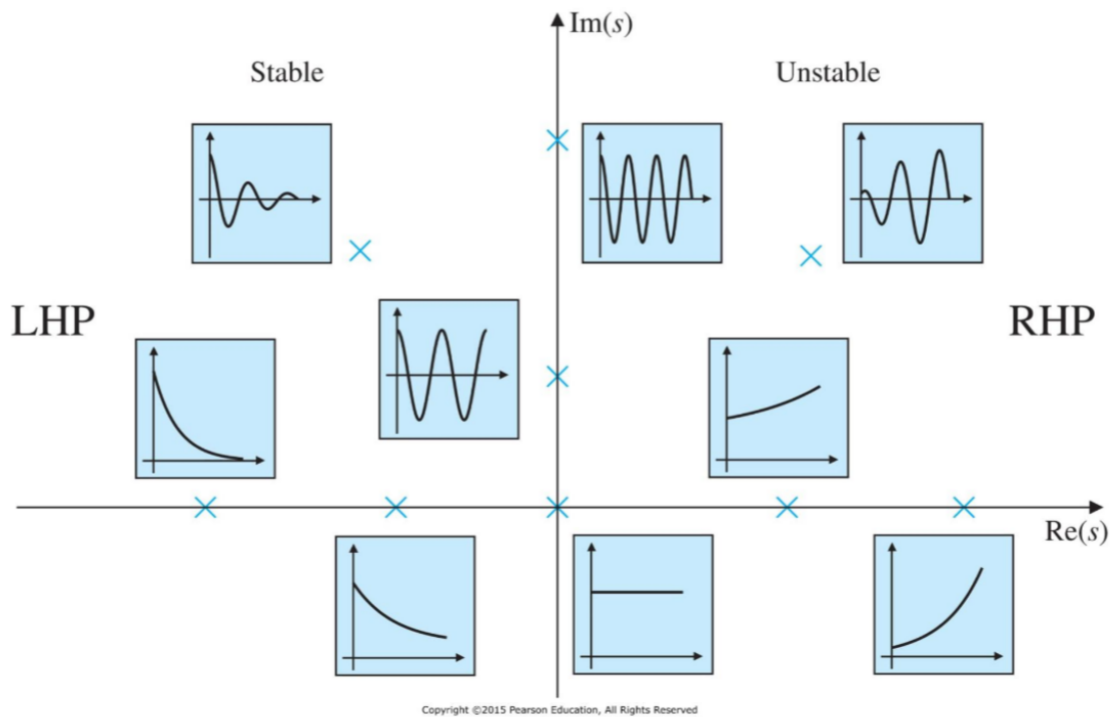
```
1 figure(1)
2 omega_n = 2*pi;
3
4 for zeta=0:0.2:1.2
5     Y = tf(omega_n^2, [1 2*zeta*omega_n omega_n^2]);
6     hold on;
7     step(Y);
8 end
```

produces



We can graph the impulse response of a system for various values of s :

Figure 3.16 Time functions associated with points in the s -plane (LHP, left half-plane; RHP, right half-plane)



with the LHS being the stable region and the RHS being the unstable region. Notice also that the negative imaginary axis is the conjugate of the positive imaginary axis so the graph is symmetric.

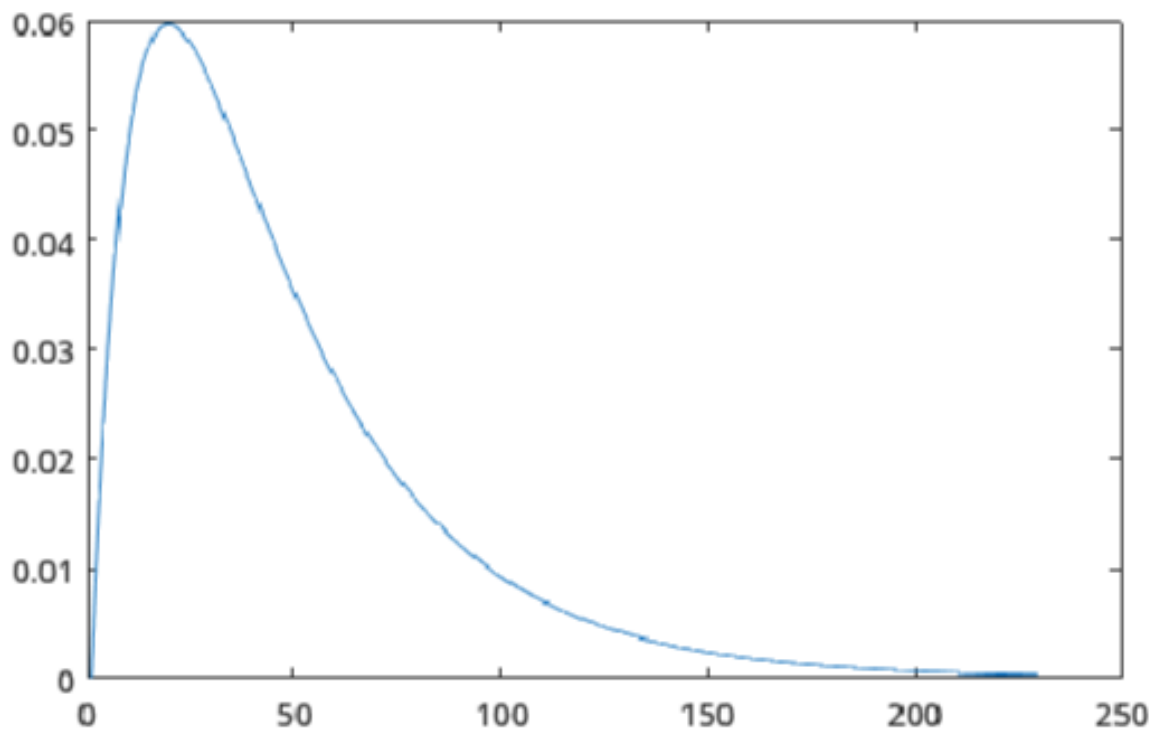
This leads to the mantra *the poles govern the system behavior*. This will be essential when we start designing controllers for systems.

Example: Consider the system

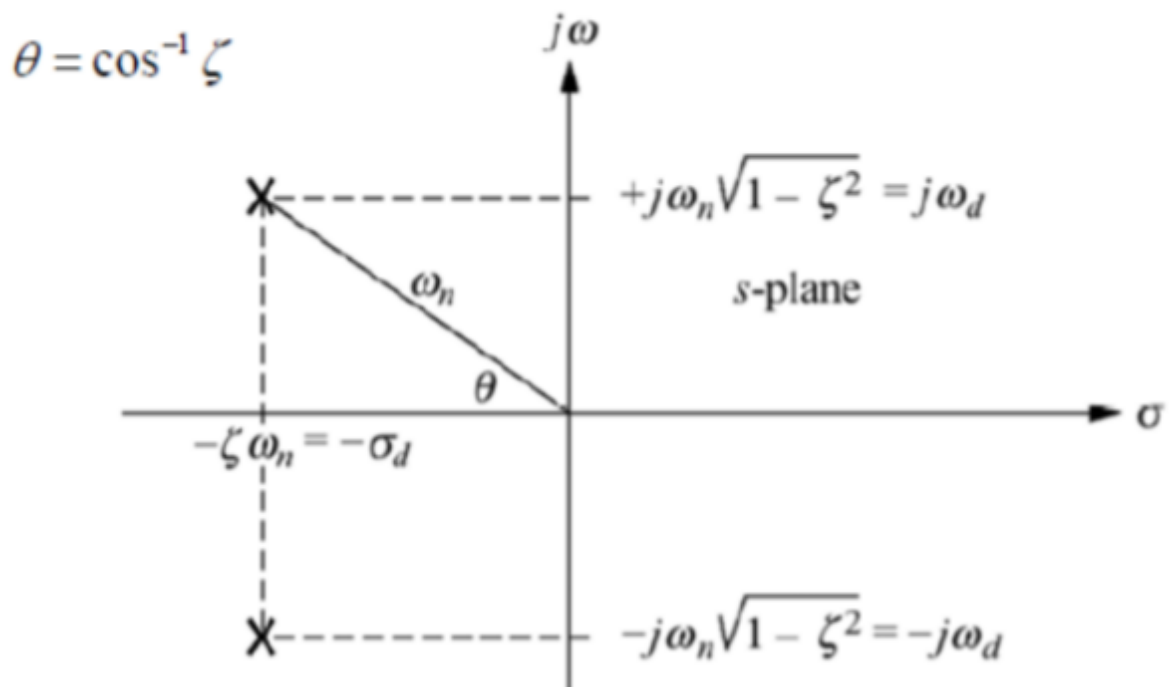
$$\frac{1}{(s + 3)(s + 10)}$$

We have poles at $s = \{-3, -10\}$ so by intuition, the system is stable.

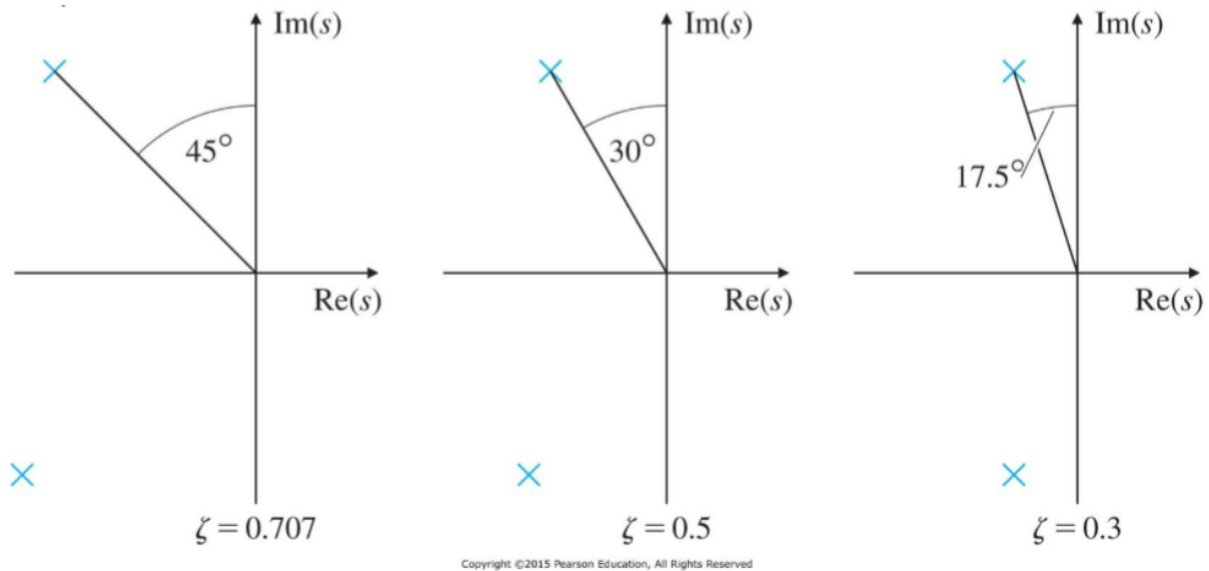
And as expected, verifying the impulse response in Matlab, we get the following graph:



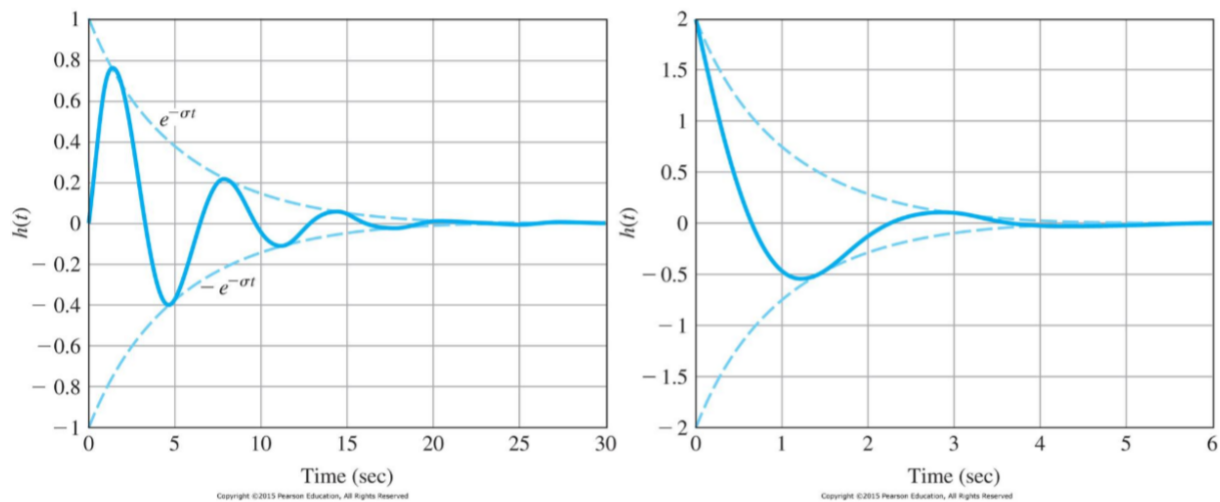
Pole location as a function of ζ :



so for example



Notice the quantity $\sigma = -\zeta\omega_n$. This is the real part of the pole and is called the *decay rate*. We can draw the *decay envelope* of the system as $\pm e^{-\sigma t}$



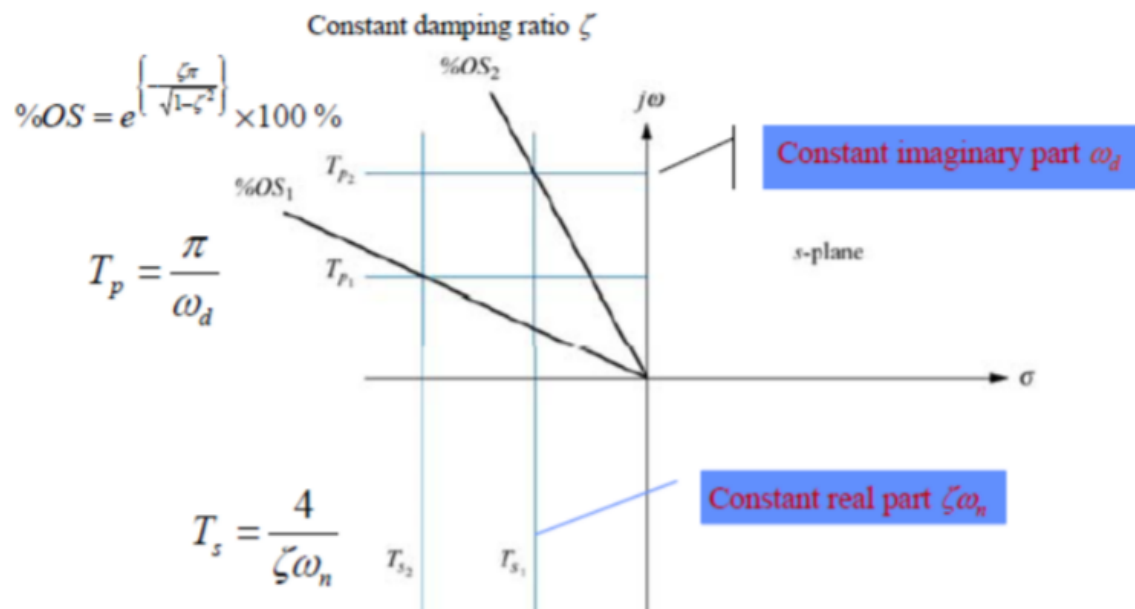
Specifications of a second order system:

- Peak time T_p : time to first peak (if $\zeta < 1$)
- Rise time T_r
- Settling time T_s
- Percent Overshoot (OS% / M_p)

$$M_p = \frac{y_{max} - y_{ss}}{y_{ss}} \cdot 100\%$$

T_r , T_p , and ω_n affect the speed of convergence while M_p and ζ affect the stability. Naturally, T_r and T_p increase together.

We can also go the other way and find the damping ratio and natural frequency from the shape of the response. This can give us **contours on the s-plane**:



Example:

$$G(s) = \frac{K}{s^2 + 4s + K} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

For any gain K , we know:

- $\zeta\omega_n = 2$ rad/s
- settling time $T_s = 4/(\zeta\omega_n) = 2$ s

If we set $K = 16$, then $\omega_n = 4$ rad/s and $\zeta = 0.5$.

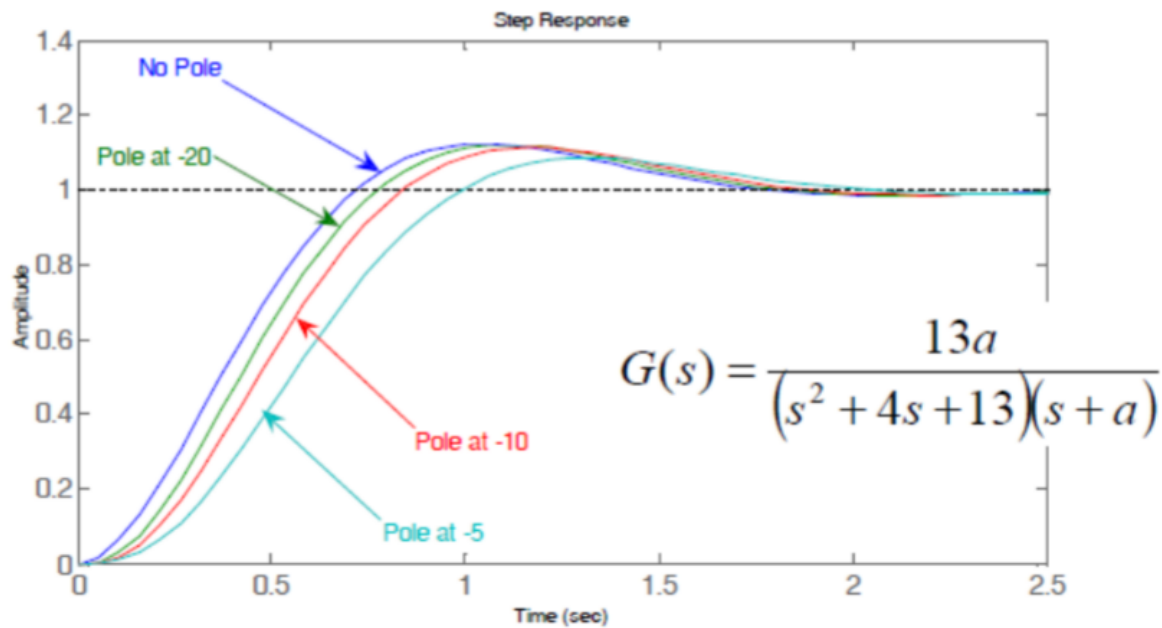
Increasing K

1. decreases ζ
2. increases OS%
3. increases ω_n and ω_d (decreases T_p and T_r) leading to a faster response

Third order system:

Adding a pole will

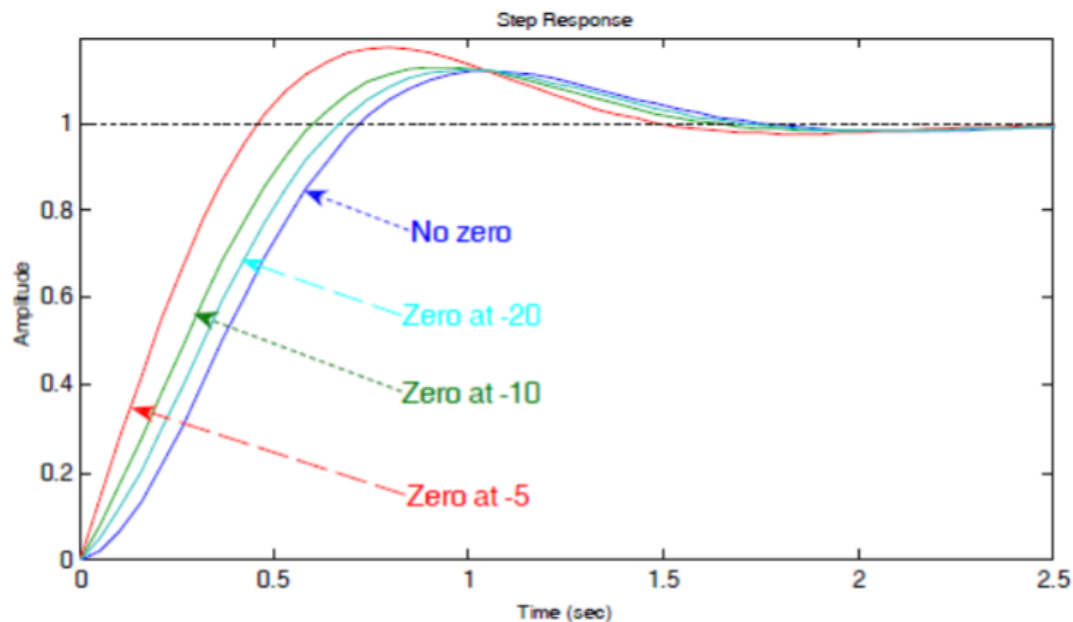
1. Slow the system response
2. Decrease overshoot



Adding zeros will

1. Speed up the system response
2. Increase overshoot
3. Complex zero can cause undershoot

$$G(s) = \frac{(13/a)(s + a)}{s^2 + 4s + 13}$$



The effects of poles and zeros are reduced as they move further to the LHP of the s-plane. Zeros can also be used to cancel out poles.

RHP zeros are called **non-minimum phase zeros** and cause unstable undershoot.

Dominant poles: quality of systems with both a 2nd order pair and other zeros/poles which are all stable (or

where they are so far LHP that they don't affect the system response).

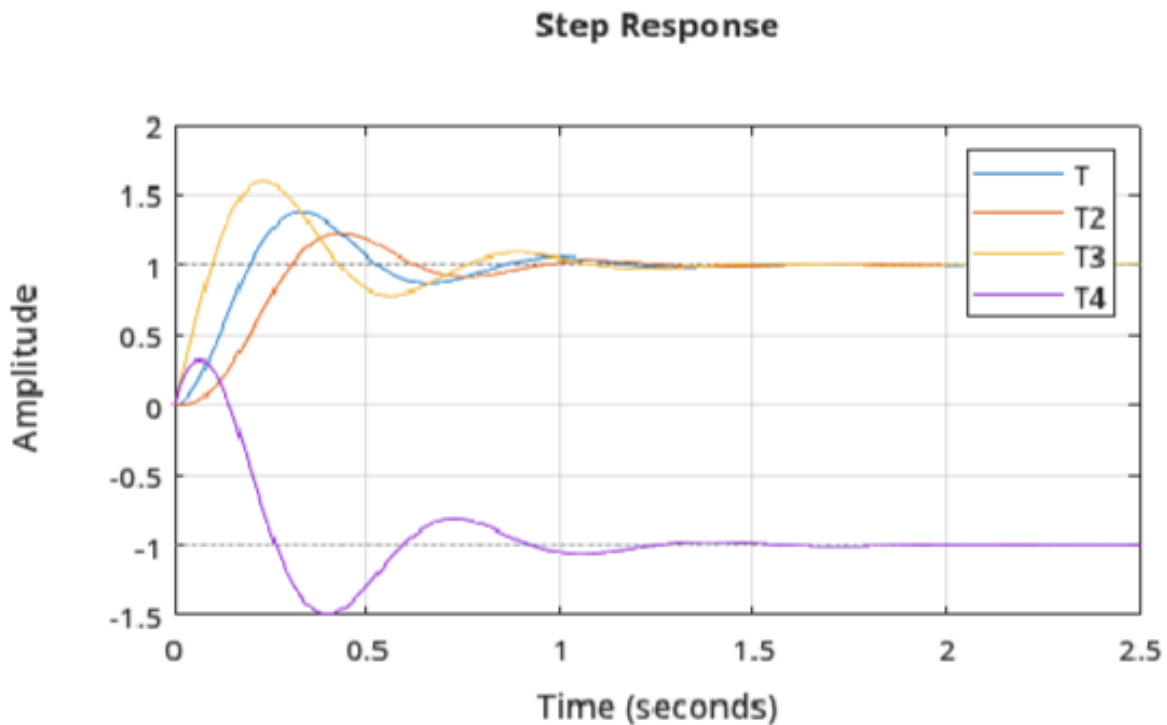
The time response of the system is *dominated* by the 2nd order pair.

Example: Consider a system with poles $s_{1,2} = -5 \pm 3j$, $s_3 = -25$. The pole at -25 is greater than 5 times the real-part of the 2nd order pole and decays so fast that it has no effect on the system response.

As a rule of thumb, *if the pole is 3-5 times greater than lowest real-part poles, they can be ignored.*

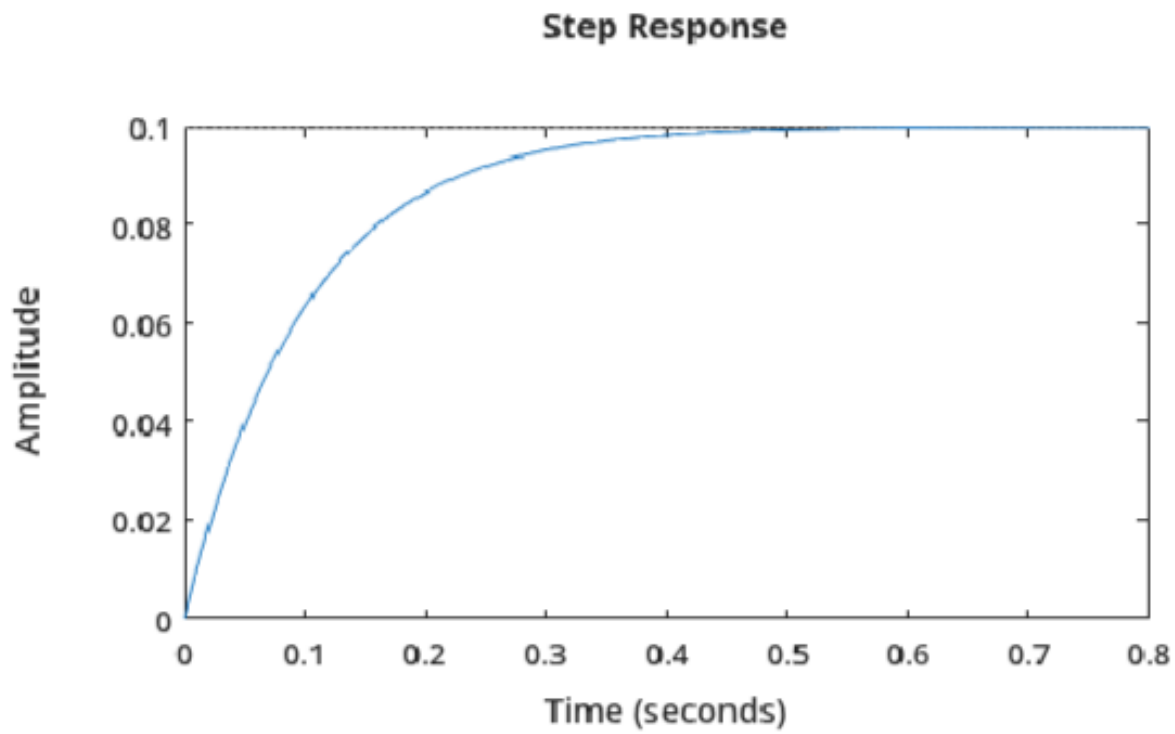
Matlab Example:

```
1  s = tf('s');
2
3  T = 100/(s^2 + 6*s + 100);
4  T2 = T * 1/(0.1*s + 1); % add a pole
5  T3 = T * (0.1*s + 1); % add a zero
6
7  T4 = T * (0.1*s - 1); % switch directions
8
9  figure(1)
10 step(T, T2, T3, T4)
11 grid
12 legend
13 shg
```



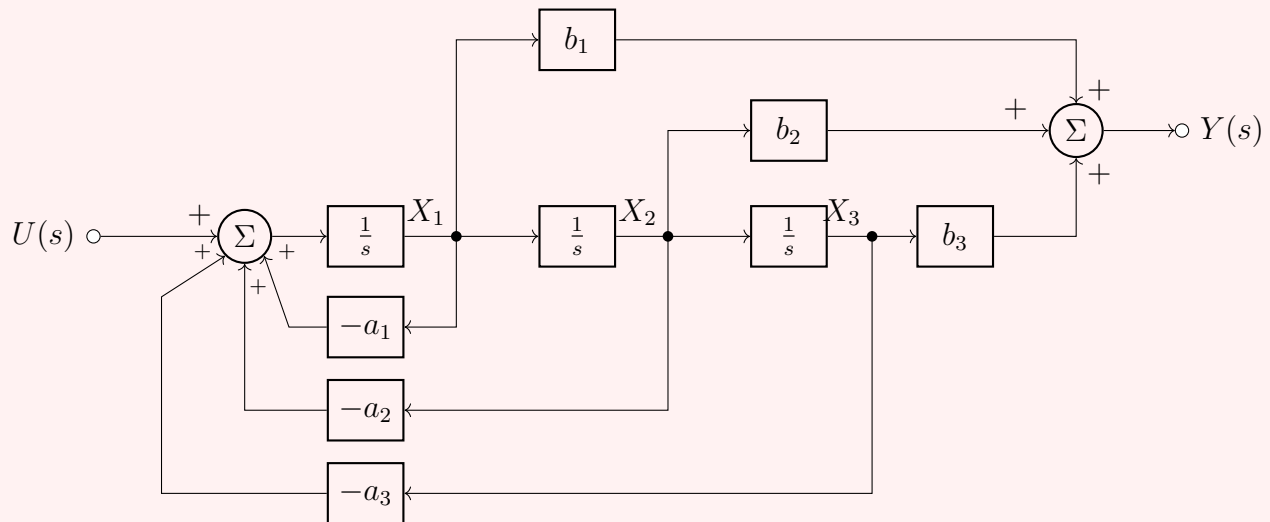
Example:

```
1  s = tf('s');
2  Tc = (s + 10) * (s + 5) / ((s^2 + 20*s + 100) * (s + 5)) % a complicated TF
3  step(Tc)
```



Notice that even though we had a step response and the system is stable, the response does not converge to 1. This is because the gain (the zero order terms) are on the order of $50/500 = 0.1$, giving a **steady-state error**.

Exercise:



(See HW 2 Problem 1)